Person-Name Extraction With Support Vector Machines:

Applying the CoNLL NER Corpus to the BNC Baby Corpus

Faith James

[Department], [University]

COSC 645

Dr. Huang

November 2, 2026

Person-Name Extraction With Support Vector Machines

Named entity recognition (NER) is the task of finding spans of text that name real-world things and labeling each span with a type such as person, location, or organization (Nadeau & Sekine, 2007). This report describes how a support vector machine (SVM) trained on *Conll_NER_corpus.txt* can identify person names in the first 10,000 sentences of the BNC Baby corpus, and reports results from a working implementation.

Named Entities and the Training Data

Conll_NER_corpus.txt is derived from the Annotated Corpus for Named Entity Recognition on Kaggle (Walia, n.d.) and follows the CoNLL column format (Tjong Kim Sang & De Meulder, 2003): one token per line, a tab, and the token's tag, with a blank line between sentences. The file holds 47,959 human-tagged sentences and 1,048,575 tokens. Tags use the BIO scheme (Ramshaw & Marcus, 1995): *B-* marks the first word of an entity, *I-* marks a word inside it, and *O* marks a word outside any entity. Person names are tagged *B-per* (16,990 tokens) and *I-per* (17,251 tokens); in this corpus a title such as *President* or *Mr.* is part of the name, so "President Mahmoud Ahmadinejad" is tagged B-per I-per I-per. All other tags (geo, org, gpe, tim, art, eve, nat) are mapped to O, leaving three classes: B-PER, I-PER, and O.

Why an SVM

An SVM finds the hyperplane that separates two classes with the largest margin (Cortes & Vapnik, 1995). Linear SVMs handle the very large, sparse feature vectors produced by text well and were a strong NER baseline before neural models (Isozaki & Kazawa, 2002). With three tags, one binary SVM is trained per tag (one-versus-rest) and each token receives the tag with the highest score. The implementation uses the LIBLINEAR solver (Fan et al., 2008)

through scikit-learn's *LinearSVC* (Pedregosa et al., 2011), and NLTK's *BNCCorpusReader* to read the BNC XML files (Bird et al., 2009).

Procedure

The procedure has two phases, shown in Figures 1 and 2. In training (Figure 1), every token becomes a feature dictionary describing the word itself (lowercased form, first and last three letters, capitalization, all-caps, digits, a trailing period as in *Mr.*), its shape (e.g., *McCain* becomes *XxXx*), and the same information for the two words on each side, since context such as *said* or a preceding title signals a name. A vectorizer builds a sparse matrix, 10% of sentences are held out to measure precision and recall, and the final model is retrained on all sentences with balanced class weights, because O covers about 97% of tokens.

In extraction (Figure 2), the first 10,000 sentences (<*s*> units) of BNC Baby are read in file order, given the same features, and tagged; each B-PER and its following I-PER tokens form a candidate name. A regular expression keeps only spans that look like names (capitalized words, initials, hyphens, apostrophes, and inner particles such as *van* or *de*). Names are written with their sentence numbers, followed by a frequency list.

Pseudocode

```
PROCEDURE TrainPersonTagger(conllFile)
    sentences ← ReadSentences(conllFile)          // split at blank lines
    FOR each (word, tag) in sentences:
        tag ← 'B-PER' if tag = 'B-per', 'I-PER' if tag = 'I-per', else 'O'
    X, y ← [], []
    FOR each sentence s, FOR i ← 0 TO |s|-1:
        X.append(Features(s, i));  y.append(tag of s[i])
    vectorizer ← DictVectorizer().fit(X)
    (Xtr, ytr), (Xte, yte) ← Split(X, y, 90% / 10%)
    svm ← LinearSVC(C = 0.1, class_weight = balanced).fit(Xtr, ytr)
    Report precision, recall, F1 of svm on (Xte, yte)
    RETURN LinearSVC(...).fit(X, y), vectorizer  // retrain on all data

FUNCTION Features(s, i)
    f ← {lower(s[i]), shape(s[i]), prefix3, suffix3, isTitle, isUpper,
         isDigit, endsWithPeriod, isFirstWord}
    FOR k IN {-2, -1, +1, +2}:
        f ← f U {lower(s[i+k]), shape(s[i+k]), isTitle(s[i+k])} or <pad>
    RETURN f
```

```
PROCEDURE ExtractPersons(svm, vectorizer, bncDir, N = 10000)
    sents ← first N sentences of BNCCorpusReader(bncDir).sents()
    results ← []
    FOR k, s IN enumerate(sents):
        tags ← svm.predict(vectorizer.transform(Features(s, i) for each i))
        FOR span IN JoinSpans(s, tags):     // B-PER followed by I-PER*
            IF Matches(NAME_REGEX, span): results.append((k, span))
    Write results and name frequencies to person_names.txt
```

Figure 1
Training the SVM Person-Name Tagger on the CoNLL Corpus

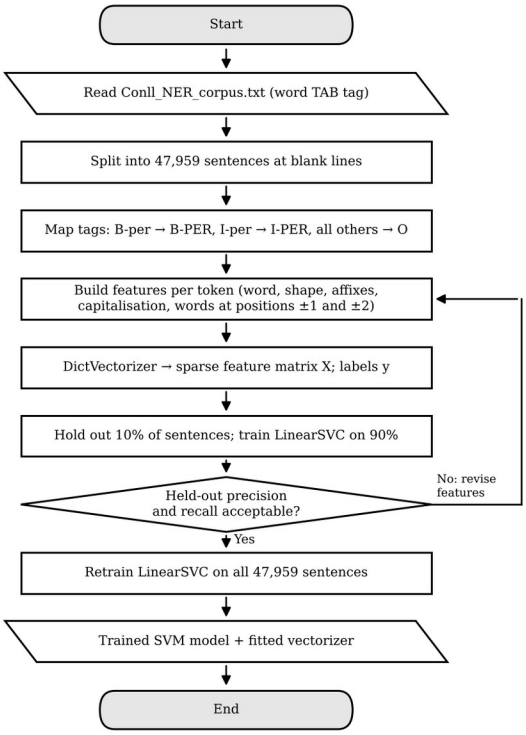
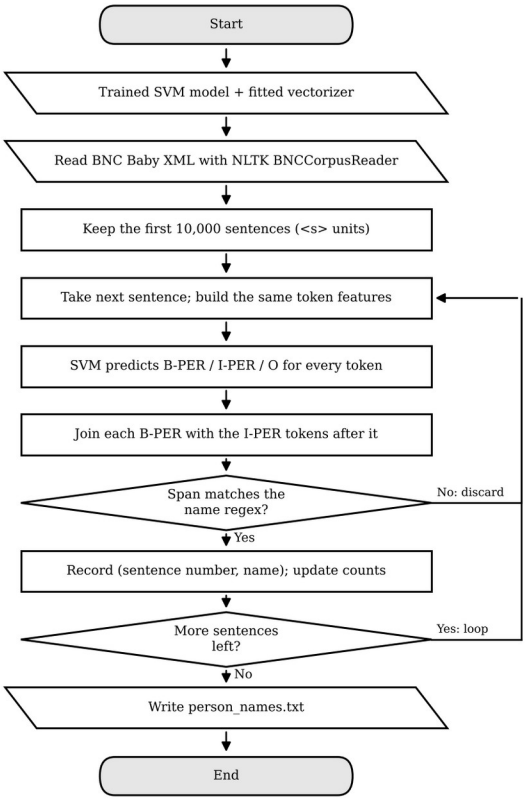


Figure 2
Extracting Person Names From the First 10,000 BNC Baby Sentences



Results

On the 10% held-out split, token-level scores for B-PER were precision .766, recall .876, and *F*1 .817; for I-PER they were .791, .926, and .853. Applied to the first 10,000 BNC Baby sentences (eight academic texts), the program extracted 1,213 person-name mentions covering 686 distinct names. The most frequent were Levin (20 mentions), Kahlo (16), Debord (16), Wilson (13), and Lord Scarman (13), alongside false positives such as *Second* (23), *GIS* (12), and *Monte Carlo* (12). As the lab allows, these errors were not corrected.

References

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python.* O'Reilly Media.

BNC Consortium. (2005). *The BNC Baby* (2nd ed.) [Data set]. Oxford Text Archive. http://hdl.handle.net/20.500.12024/2553

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., & Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research, 9*, 1871–1874.

Isozaki, H., & Kazawa, H. (2002). Efficient support vector classifiers for named entity recognition. In *Proceedings of the 19th International Conference on Computational Linguistics* (pp. 390–396). Association for Computational Linguistics. https://doi.org/10.3115/1072228.1072282

Nadeau, D., & Sekine, S. (2007). A survey of named entity recognition and classification. *Lingvisticae Investigationes, 30*(1), 3–26. https://doi.org/10.1075/li.30.1.03nad

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Ramshaw, L. A., & Marcus, M. P. (1995). Text chunking using transformation-based learning. In *Proceedings of the Third Workshop on Very Large Corpora* (pp. 82–94). Association for Computational Linguistics.

Tjong Kim Sang, E. F., & De Meulder, F. (2003). Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003* (pp. 142–147). Association for Computational Linguistics.

Walia, A. (n.d.). *Annotated corpus for named entity recognition* [Data set]. Kaggle. Retrieved November 1, 2026, from https://www.kaggle.com/abhinavwalia95/entity-annotated-corpus